

# MERCAHASH V1

## Technical Overview of Mercatura's Memory-Hard Proof-of-Work

---

Authoritative 128 MiB profile • Mercatura Core

Technical note • Version 1.0 • October 2026

### Executive summary

MercaHash V1 is Mercatura's consensus proof-of-work function. It accepts the canonical 80-byte block header, derives a 1,024-bit working state, fills and mutates a private 128 MiB scratchpad, performs data-dependent memory mixing with periodic cross-lane and SHA3-512 checkpoint mixing, samples the resulting memory state, and produces a 256-bit SHA3-256 proof-of-work value.

The design goal is not to make specialized hardware impossible. Rather, MercaHash deliberately raises the importance of large mutable memory, irregular data-dependent access, serial state evolution, and general-purpose integer execution so that commodity CPUs remain meaningfully competitive and the advantage of highly specialized hardware is harder to obtain cheaply.

MercaHash is intentionally separated from Mercatura's block identity. The Bitcoin-style SHA256d header hash remains the block identifier used by indexes, RPCs, locators, previous-block references, and explorer-visible block hashes. MercaHash is used specifically for proof-of-work mining and validation.

**Scope.** This document explains the current MercaHash V1 design and implementation at a technical level. It describes the algorithm's structure, parameters, consensus role, performance implications, and security rationale; it is not a formal cryptographic proof of ASIC resistance.

# 1. Design goals and consensus architecture

MercaHash was designed as a CPU-oriented, memory-hard proof-of-work that stays comparatively simple to audit and integrate into a Bitcoin Core-derived codebase. The design avoids a virtual machine, JIT compiler, or randomized instruction program; instead it combines a fixed ARX state machine, a large mutable scratchpad, data-dependent memory references, and periodic SHA3 checkpoints.

## 1.1 Separation of block identity and proof of work

Mercatura deliberately gives the 80-byte block header two cryptographic interpretations. The ordinary Bitcoin-style double-SHA256 header hash remains the block's identity, while MercaHash produces the value that is compared with the proof-of-work target. This isolates the PoW change from unrelated chain semantics and avoids redefining block locators, disk/index identifiers, RPC block hashes, compact-block identity, or the previous-block linkage.

| Input                | Function          | Consensus role                    |
|----------------------|-------------------|-----------------------------------|
| 80-byte block header | SHA256d GetHash() | Block identity / chain references |
| 80-byte block header | MercaHash V1      | Proof-of-work value               |

## 1.2 Proof-of-work acceptance

Mining varies the block header (normally through the nonce and, when necessary, extranonce-driven merkle-root changes) and evaluates MercaHash for each candidate. Validation recomputes the same deterministic function. A candidate satisfies proof of work when the interpreted MercaHash value is at or below the target encoded by nBits.

$$PoW\ valid \Leftrightarrow uint256(MercaHashV1(header)) \leq Target(nBits)$$

## 1.3 Why a large private scratchpad?

Each hashing worker owns a 128 MiB mutable scratchpad. The memory is not a read-only lookup table: it is generated from the candidate header, then repeatedly overwritten as the state evolves. Consequently, a worker must carry both the large memory image and the evolving 16-lane state through the hash. The intended effect is to make raw arithmetic throughput alone insufficient; memory capacity, latency, bandwidth, and the ability to sustain irregular dependencies become important parts of mining performance.

**Important distinction.** MercaHash is memory-hard by design, but “memory-hard” does not mean “ASIC-proof.” Specialized hardware may still be built. The design objective is to narrow the economic advantage by making memory and control complexity part of the cost structure.

## 2. Frozen V1 workload and algorithm pipeline

### 2.1 Authoritative parameters

| Parameter           | MercaHash V1                                    |
|---------------------|-------------------------------------------------|
| Input header        | 80 bytes                                        |
| Output              | 32 bytes / 256 bits                             |
| Scratchpad          | 128 MiB private mutable memory                  |
| Scratchpad line     | 64 bytes = $8 \times 64$ -bit words             |
| Scratchpad lines    | $2,097,152 = 2^{21}$                            |
| Binding block       | 4 KiB = 64 lines                                |
| Binding blocks      | $32,768 = 2^{15}$                               |
| Binding passes      | 1                                               |
| Working state       | $16 \times 64$ -bit lanes                       |
| Main mixing steps   | $131,072 = 2^{17}$                              |
| Secondary touch     | Every 16 steps $\rightarrow$ 8,192 touches      |
| Cross-lane mix      | Every 64 steps $\rightarrow$ 2,048 events       |
| SHA3-512 checkpoint | Every 1,024 steps $\rightarrow$ 128 checkpoints |
| Final sampling      | 256 scratchpad reads                            |
| Finalization        | Domain-separated SHA3-256                       |

$$N_{lines} = 134,217,728 \text{ bytes} \div 64 \text{ bytes} = 2,097,152 \text{ lines}$$

### 2.2 Execution pipeline

| Stage | Name            | Purpose                                                                                                        |
|-------|-----------------|----------------------------------------------------------------------------------------------------------------|
| 1     | Initial state   | Two domain-separated SHA3-512 digests of the 80-byte header become 16 little-endian 64-bit lanes.              |
| 2     | Scratchpad fill | Generate all $2^{21}$ lines while evolving the state; V1 alternates ARX column and diagonal half-rounds.       |
| 3     | Binding pass    | Rewrite every 4 KiB block using current data, the previous block, and a pseudo-random earlier reference block. |
| 4     | Main mixing     | Perform $2^{17}$ data-dependent primary read/modify/write steps, with periodic secondary memory touches.       |
| 5     | Periodic mixing | Every 64 steps mix the full state; every 1,024 steps hash checkpoint material with SHA3-512 and feed it back.  |
| 6     | Final sample    | Perform 256 state-dependent reads from the mutated scratchpad, mixing each into the state.                     |
| 7     | Final hash      | SHA3-256 over a domain tag, the original header, and the final 16-lane state.                                  |

## 3. State initialization, ARX mixing, fill and binding

### 3.1 Header-derived state

The working state contains sixteen 64-bit unsigned lanes. MercaHash derives the first eight lanes from SHA3-512("MercaHash-v1/state/0" || header) and the second eight lanes from a separately domain-separated SHA3-512("MercaHash-v1/state/1" || header). Each 64-bit word is decoded explicitly in little-endian form.

$$S[0..7] = \text{LE64-words}(\text{SHA3-512}(D_0 \parallel \text{header})); \quad S[8..15] = \text{LE64-words}(\text{SHA3-512}(D_1 \parallel \text{header}))$$

### 3.2 The MercaARX primitive

Most non-cryptographic state diffusion uses addition-rotation-XOR (ARX) operations on unsigned 64-bit words. The core four-word G transform is applied first in columns and then in diagonals. Unsigned addition naturally wraps modulo  $2^{64}$ , giving deterministic overflow semantics.

$$a \leftarrow a + b; \quad d \leftarrow \text{ROTL}(d \oplus a, 32); \quad c \leftarrow c + d; \quad b \leftarrow \text{ROTL}(b \oplus c, 24)$$

$$a \leftarrow a + b; \quad d \leftarrow \text{ROTL}(d \oplus a, 16); \quad c \leftarrow c + d; \quad b \leftarrow \text{ROTL}(b \oplus c, 63)$$

### 3.3 Scratchpad fill

Before memory-hard mixing begins, V1 deterministically fills the entire 128 MiB scratchpad. For each 64-byte line, the line index perturbs the state, V1 performs either the column or diagonal half of the ARX round depending on line parity, and eight 64-bit words are emitted as XORs of the lower and upper halves of the state. The newly written line is then fed back into multiple state lanes. This makes later lines dependent on the evolving history rather than on an independently parallelizable per-line formula.

### 3.4 One memory-binding pass

The filled scratchpad is then rewritten in 4 KiB blocks. Each block contains 64 lines. For a block  $b$ , MercaHash derives a reference seed from the live state, the block index, and the block's first line. Except for the first block, the pseudo-random reference block is constrained to an earlier block; the immediately previous block is also read. Every line is recomputed from three sources: its current value, a line from the previous block, and a rotated line from the selected reference block, together with live state words and fixed rotations/constants.

$$\text{reference\_block}(b) < b, \quad \text{for } b > 0$$

This binding stage deliberately creates sequential inter-block dependencies before the main random-access loop. A device cannot treat the 128 MiB as a static table generated once for all nonces: the entire scratchpad is specific to the current header and is mutated again during the hash.

**Why this matters.** The fill creates header-specific memory; the binding pass makes large regions depend on prior and pseudo-random earlier regions; the main mix then destroys any assumption that the resulting memory stays read-only.

## 4. Main data-dependent mixing

### 4.1 Primary step: one mutable random touch per step

MercaHash performs 131,072 serial primary steps. At each step  $t$ , a state-derived 64-bit seed chooses one of the  $2^{21}$  scratchpad lines. Because the line count is a power of two, the implementation uses a 21-bit mask rather than a division.

$$i_p = (S_0 \oplus \text{ROTL}(S_5, 17) \oplus S_{10} \oplus \text{ROTL}(S_{15}, 41) \oplus t \cdot C_0) \& (2^{21} - 1)$$

The selected line is read, mixed into both halves of the 16-lane state, transformed into a new eight-word value, and written back to the same address. The written data then feeds back into additional lanes. Thus the next address depends on state that has already incorporated earlier memory contents and earlier writes.

### 4.2 Secondary touch every 16 steps

Every sixteenth primary step triggers a second state-dependent access. The secondary index uses different state lanes, the current step, and one word from the just-written primary line. If it accidentally selects the same line as the primary step, the implementation flips the low index bit so that the two accesses are distinct. Across V1 this produces 8,192 extra read/modify/write operations.

### 4.3 Cross-lane mixing every 64 steps

Every 64 steps the state is forced through two complete MercaARX rounds after step-dependent injection. This prevents the sixteen lanes from evolving as loosely coupled subgroups for long periods and ensures that memory observations in one region of the state influence future address generation elsewhere.

### 4.4 SHA3-512 checkpoint every 1,024 steps

Every 1,024 steps MercaHash creates a domain-separated SHA3-512 checkpoint. The checkpoint hashes the step number, the primary and secondary line indices, all sixteen state lanes, and the eight words written by each of the two most recent memory operations. The resulting 512-bit digest is folded back into the state, followed by another ARX round.

$$C_t = \text{SHA3-512}(Dcp \parallel t \parallel i_p \parallel i_s \parallel S \parallel W_p \parallel W_s)$$

The checkpoint mechanism periodically introduces a strong cryptographic compression step into the otherwise lightweight ARX/memory loop. V1 executes exactly 128 such checkpoints. The intent is to make long-range state evolution harder to shortcut or model with a simplified arithmetic pipeline while keeping the dominant cost in the large scratchpad.

| Event                    | Frequency         | V1 count |
|--------------------------|-------------------|----------|
| Primary mutable access   | Every step        | 131,072  |
| Secondary mutable access | Every 16 steps    | 8,192    |
| Full cross-lane event    | Every 64 steps    | 2,048    |
| SHA3-512 checkpoint      | Every 1,024 steps | 128      |

## 5. Final sampling, output, and mining behavior

### 5.1 Final scratchpad sampling

After the main loop, MercaHash does not immediately hash the state. It performs 256 additional reads from the final mutated scratchpad. Each sample index is derived from the current state and the sample counter using the same 21-bit line mask. The sampled line updates both halves of the state; every sixteenth sample performs a full ARX round.

$$i_{\text{sample}} = (S_0 \oplus \text{ROTL}(S_5, 17) \oplus S_{10} \oplus \text{ROTL}(S_{15}, 41) \oplus \text{sample} \cdot C_0) \& (2^{21} - 1)$$

### 5.2 Domain-separated finalization

The final 256-bit output is SHA3-256 over the finalization domain string, the original 80-byte header, and all sixteen state lanes serialized as little-endian 64-bit words. The scratchpad itself is not appended to the final hash; instead, its influence has already been absorbed through the fill, binding, main mixing, checkpoint, and final-sampling stages.

$$H_{\text{PoW}} = \text{SHA3-256}(D_{\text{final}} \parallel \text{header} \parallel \text{LE64}(S_0) \parallel \dots \parallel \text{LE64}(S_{15}))$$

### 5.3 Memory and throughput implications

A mining process normally allocates one 128 MiB scratchpad per active hashing worker and reuses that allocation across successive nonces. Keeping allocation outside the consensus hash function avoids turning allocator cost into part of the benchmark and allows miners to choose a worker count appropriate for their memory subsystem.

$$\text{Scratchpad memory} \approx 128 \text{ MiB} \times \text{active hashing workers}$$

For example, six workers require about 768 MiB of scratchpad memory before miner, operating-system, and node overhead. Increasing worker count therefore eventually becomes limited not only by CPU cores but also by memory bandwidth, cache behavior, and system responsiveness.

### 5.4 Reference development performance

A development benchmark on the Ryzen 5 4600G / Ubuntu 22.04 WSL2 system recorded approximately 113.69 ms per MercaHash (about 8.8 H/s) in a measured configuration. Aggregate miner throughput depends on thread count, scheduling, and the memory subsystem, so this figure is a reference point rather than a protocol guarantee.

### 5.5 Mining and validation must be identical

MercaHash V1 is a consensus function. A miner that computes a subtly different hash is simply mining invalid blocks. Mercatura therefore uses the same frozen HashV1 workload for mining and validation, and production callers do not pass benchmark-time workload parameters. Header synchronization must also verify actual MercaHash proof of work rather than trusting claimed chainwork or a SHA256d block identifier.

**Operational consequence.** Changing any V1 workload parameter—scratchpad size, mix count, rotations, constants, fill mode, binding passes, checkpoint schedule, serialization, or finalization—changes consensus proof of work and therefore requires a deliberate network upgrade rather than a miner-only optimization.

## 6. Security rationale, limitations, and implementation rules

### 6.1 What MercaHash is trying to make expensive

- Memory capacity per parallel worker: 128 MiB must be available to each active hash context.
- Irregular mutable access: addresses depend on evolving state, and accessed lines are rewritten rather than merely read.
- Serial dependence: later addresses and values depend on state that incorporates earlier reads and writes.
- Cross-state coupling: frequent full-state ARX rounds prevent long-lived independent lane groups.
- Cryptographic checkpointing: periodic SHA3-512 compression couples indices, state, and recent writes.
- Recomputation shortcuts: header-specific fill and a binding pass make the scratchpad expensive to recreate or treat as a reusable static table.

### 6.2 What MercaHash does not claim

MercaHash is not presented as mathematically ASIC-proof or GPU-proof. Specialized implementations may discover architectural advantages, better memory scheduling, larger bandwidth, or custom logic. The relevant question is economic: whether those advantages are large enough to centralize mining materially. That question must be answered by continued CPU/GPU/FPGA/ASIC benchmarking and real network observation, not by the algorithm's label.

### 6.3 Determinism and consensus safety

Consensus execution is defined using fixed-width unsigned integer operations, explicit rotations, explicit little-endian serialization, and fixed iteration counts. Floating-point arithmetic is excluded. Undefined or platform-dependent behavior is unacceptable because every conforming node must produce the same 256-bit output for the same 80-byte header.

- All state arithmetic uses fixed-width integer semantics with intentional modulo- $2^{64}$  wraparound where applicable.
- All scratchpad reads and writes operate on explicitly little-endian 64-bit words.
- Production proof of work uses the frozen HashV1 entry point rather than caller-supplied tuning parameters.
- Block identity remains SHA256d; MercaHash is used only for proof-of-work computation and verification.
- Permanent test vectors and cross-platform tests are essential safeguards against compiler or architecture divergence.

### 6.4 Design trade-off

MercaHash deliberately occupies a middle ground. It is substantially more memory- and state-intensive than a simple cryptographic compression-loop PoW, but it avoids the VM/JIT complexity of RandomX-style randomized instruction execution. That choice favors a smaller consensus attack surface, easier deterministic testing, and clearer integration into Mercatura, while accepting that specialized hardware resistance may be less absolute than the most complex CPU-oriented schemes.

## Conclusion

The defining feature of MercaHash V1 is not any single primitive; it is the composition of a large header-specific mutable memory image, serial data-dependent access, mandatory write-back, repeated cross-lane diffusion, periodic SHA3 checkpoints, and deterministic final sampling. Together these features make proof-of-work performance depend on a broad set of resources that commodity CPUs already possess, while keeping the consensus algorithm fixed, auditable, and reproducible.

## Technical basis

Prepared from the current Mercatura specification and implementation: Mercatura Spec and Development Roadmap v1.4 (§1.4 Proof of Work: MercaHash), `src/crypto/mercahash.h`, `src/crypto/mercahash.cpp`, and the MercaHash consensus-integration handoff notes. Parameter values shown here reflect the authoritative MercaHash V1 128 MiB profile.